SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA0304	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	P. Dada Kalander	**Reg. No.:**	192424272
Section / Batch:	2024	**Date:**	13-07-2026

Code of Conduct

I P.Divakaran (192424260) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.
Identify email addresses.
Extract mobile numbers.
Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
Extract years of experience.
Generate a structured summary of the candidate's profile.
Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.
Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. **Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:**

Aim

To design and implement a Python-based Resume Information Extraction System using Regular Expressions (Regex) to extract candidate details, identify eligible candidates, and generate a structured profile summary.

Problem Statement

A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.
The system should perform the following tasks:

- Extract the candidate's name.
- Identify email addresses.
- Extract mobile numbers.
- Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
- Extract years of experience.
- Generate a structured summary of the candidate's profile.

Algorithm

1. Import the re (Regular Expression) module.
2. Store the resume text in a string.
3. Extract the candidate's name using Regex.
4. Extract the email address.
5. Extract the mobile number.
6. Search for the required technical skills.
7. Extract the years of experience.
8. Display the extracted information in a structured format.
9. Check whether the candidate has at least **2 years of experience** and possesses the **Python** skill.
10. Display the eligibility status.

Python Program

```
import re
# Sample Resume
resume = """ Name:
Priya Nair
Email: priya.nair@outlook.com Mobile:
+91 9845123670

Skills:
Python, Java, SQL, Machine Learning, NLP Experience: 4
years
Worked as a Data Analyst. """

# Extract Name
name = re.search(r"Name\s*:\s*([A-Za-z ]+)", resume) candidate_name =
name.group(1) if name else "Not Found"

# Extract Email
email = re.search(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", resume) candidate_email =
email.group() if email else "Not Found"
```

```
    # Extract Mobile Number
    mobile = re.search(r"(\+91[- ]?)?[6-9]\d{9}", resume) candidate_mobile =
    mobile.group() if mobile else "Not Found

# Detect Skills
    skills = ["Python", "Java", "SQL", "Machine Learning", "NLP"] found_skills = []

    for skill in skills:
        if re.search(skill, resume, re.IGNORECASE):
            found_skills.append(skill)

    # Extract Experience
    experience = re.search(r"(\d+)\s+years?", resume, re.IGNORECASE) years =
    int(experience.group(1)) if experience else 0

    # Display Summary
    print("========== Candidate Profile ==========")
    print("Name                :", candidate_name) print("Email :",
    candidate_email) print("Mobile    :", candidate_mobile)
    print("Skills              :", ", ".join(found_skills))
    print("Experience :", years, "Years")

    # Eligibility Check print("\nEligibility
    Status")

    if years >= 2 and "Python" in found_skills: print(candidate_name, "is
        ELIGIBLE for Shortlisting.")
    else:
        print(candidate_name, "is NOT ELIGIBLE."
```

Output

========== Candidate Profile

========== Name : Priya Nair
Email :
priya.nair@outlook.com Mobile
: +91 9845123670
Skills : Python, Java, SQL, Machine Learning,
NLP Experience : 4 Years

Eligibility Status

Priya Nair is ELIGIBLE for Shortlisting.

Regular Expressions Used

Field	Regular Expression	Purpose
Name	**Name\s*:\s*([A-Za-z]+)**	**Extract candidate name**
Email	**[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}**	**Extract email address**
Mobile	**(\+91[-]?)?[6-9]\d{9}**	**Extract Indian mobile number**
Experience	**(\d+)\s+years?**	**Extract years of experience**
Skills	**Python, Java, SQL, Machine Learning, NLP**	**Detect technical skills**

Result

The Resume Information Extraction System was successfully implemented using Python and Regular Expressions. The program accurately extracted the candidate's name, email address, mobile number, technical skills, and years of experience. It generated a structured profile summary and correctly identified whether the candidate met the eligibility criteria of minimum 2 years of experience and Python skill for shortlisting.

2. Product Search System Using Regular Expressions

Aim

To design and implement a Python-based Product Search System using Regular Expressions (Regex) for efficient and flexible product searching in an e-commerce application.

Problem Statement

An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.
The system should perform the following tasks:

- Search products using an exact keyword.
- Perform prefix-based searches.
- Perform suffix-based searches.
- Search using partial keywords.
- Perform case-insensitive searches.
- Display all matching product names.
- Generate a report showing the total number of matching products for each search.

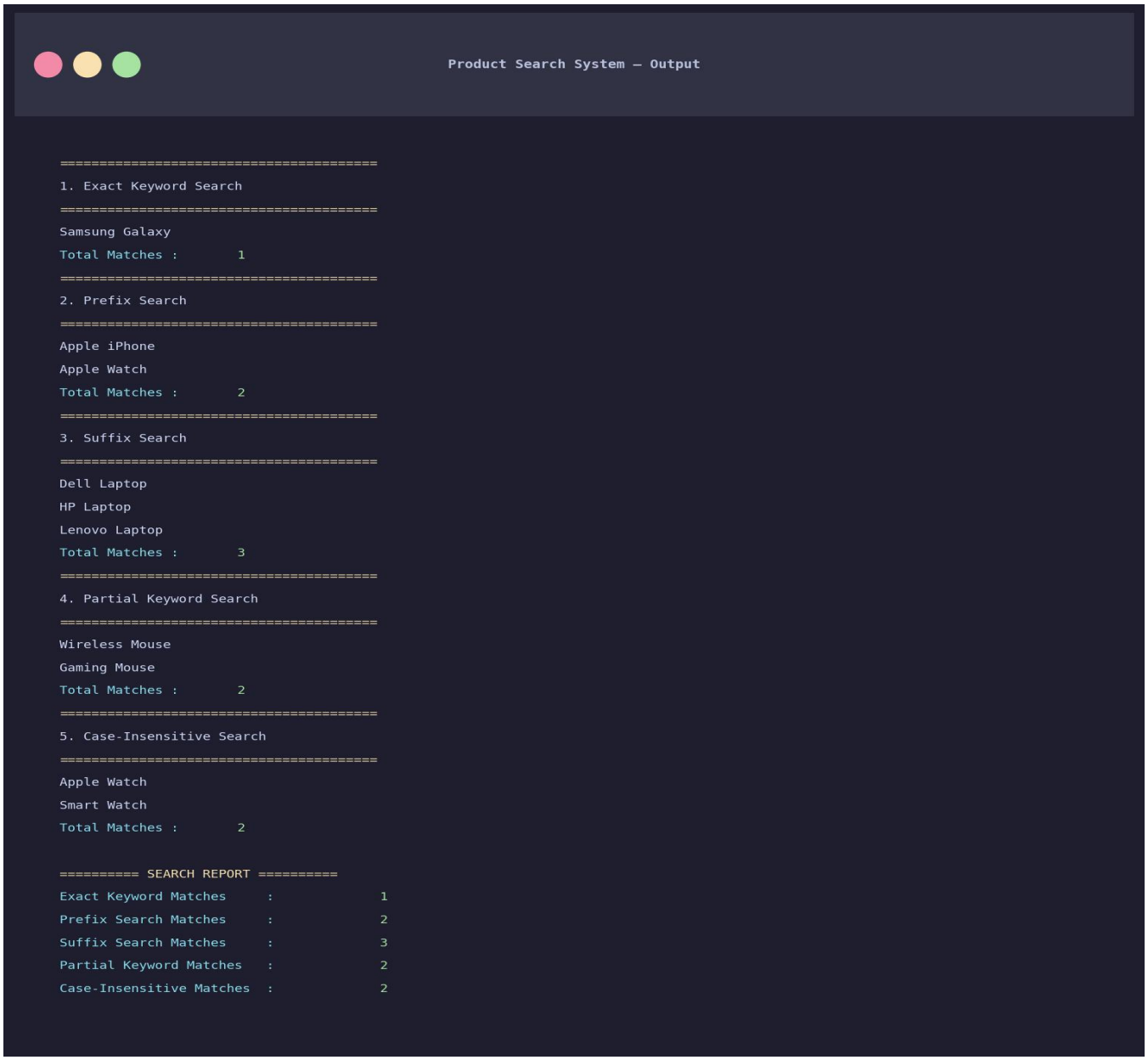
Algorithm

1. Import the re module.
2. Create a list of product names.
3. Accept different search patterns.
4. Perform:
    - Exact keyword search.
    - Prefix search.
    - Suffix search.
    - Partial keyword search.
    - Case-insensitive search.
5. Display all matching products.
6. Count the number of matching products.
7. Display the search report.

Python Program

```
import re

# Product List
products = [
    "Apple iPhone",
    "Apple Watch",
    "Samsung
    Galaxy",
    "Samsung TV",
    "Dell Laptop",
    "HP Laptop",
    "Lenovo Laptop",
    "Wireless Mouse",
    "Gaming Mouse",
    "Bluetooth
    Speaker", "Smart
    Watch", "Phone
    Charger"
```

```
]

# Function to perform search
def search_products(pattern, description, flags=0):
    matches = [product for product in products if re.search(pattern, product, flags)]

    print("\n",
    "="*40)
    print(descriptio
    n)
    print("="*40)

    if matches:
        for product in matches:
            print(product)
    else:
        print("No matching products found.")

    print("Total Matches :", len(matches))
    return len(matches)

# Exact Keyword Search
exact = search_products(r"^Samsung Galaxy$", "1. Exact Keyword Search")

# Prefix Search
prefix = search_products(r"^Apple", "2. Prefix Search")

# Suffix Search
suffix = search_products(r"Laptop$", "3. Suffix Search")

# Partial Keyword Search
partial = search_products(r"Mouse", "4. Partial Keyword Search")

# Case-Insensitive Search
case = search_products(r"watch", "5. Case-Insensitive Search", re.IGNORECASE)

# Search Report
print("\n========== SEARCH REPORT ==========")
print("Exact Keyword Matches
                                    :",
exact) print("Prefix Search Matches
                                    :",
prefix) print("Suffix Search Matches
                                    :", suffix)
print("Partial Keyword Matches
                                    :",
partial) print("Case-Insensitive
Matches  :", case)
```



Sample Output

==========

1. Exact Keyword Search

==========

Samsung
Galaxy Total
Matches : 1

==========

2. Prefix Search

==========

Apple iPhone
Apple Watch
Total Matches :
2

==========
3. Suffix Search
==========

Dell Laptop
HP Laptop
Lenovo
Laptop
Total Matches : 3

==========
1. Partial Keyword Search
==========
Wireless Mouse
Gaming Mouse
Total Matches :
2

==========
2. Case-Insensitive Search
==========
Apple Watch
Smart Watch
Total Matches :
2

========== SEARCH REPORT ==========
Exact Keyword Matches 1
Prefix Search Matches 2
Suffix Search Matches 3
Partial Keyword Matches 2
Case-Insensitive Matches 2

Regular Expressions Used

Search Type	Regular Expression	Purpose
Exact Keyword	^Samsung Galaxy$	Matches the complete product name exactly
Prefix Search	^Apple	Matches products starting with "Apple"
Suffix Search	Laptop$	Matches products ending with "Laptop"
Partial Search	Mouse	Matches products containing "Mouse"
Case-Insensitive	watch with re.IGNORECASE	Matches "Watch" regardless of letter case

Result

The Product Search System was successfully implemented using Python and Regular Expressions. The program performed exact, prefix, suffix, partial, and case-insensitive searches, displayed all matching product names, and generated a report showing the total number of matching products for each search type. This approach improves the flexibility and efficiency of product searching in an e-commerce environment.

3. Student Registration Validation System Using Regular Expressions

Aim

To design and implement a Python-based Student Registration Validation System using Regular Expressions (Regex) to verify student information before course enrollment.

Problem Statement

A university is developing an automated online registration portal to verify student information before course enrollment.

The system should perform the following tasks:

- Validate the student register number.
- Validate the institutional email address.
- Validate the course code.
- Validate the semester information.
- Validate the student's mobile number.
- Display appropriate validation messages for each field.
- Generate a final registration status report indicating whether the registration is successful.

Algorithm

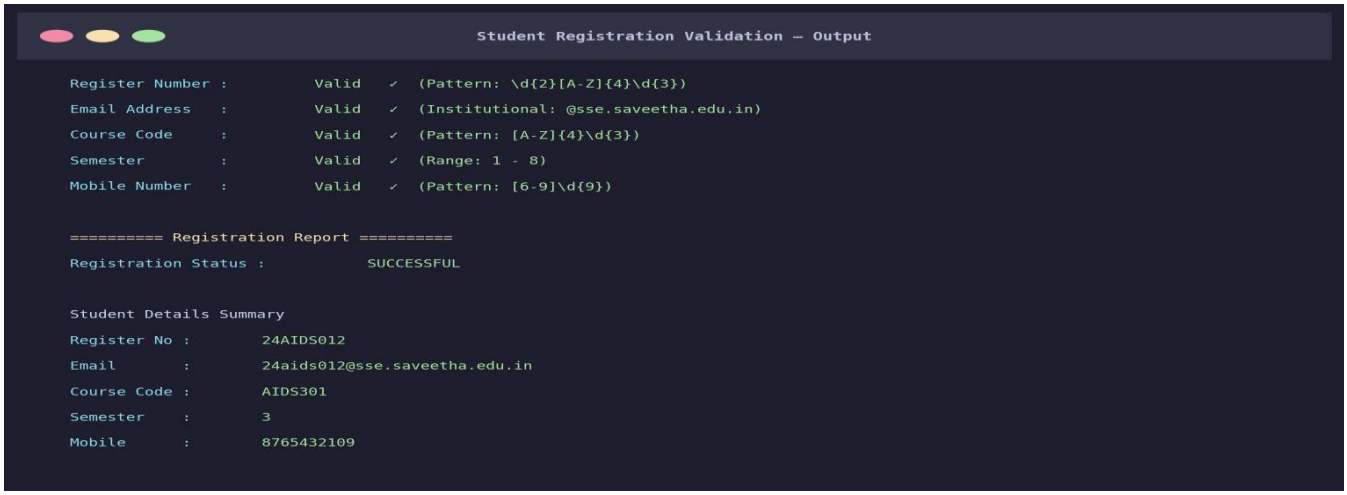
1. Import the re module.
2. Read the student's registration details.
3. Validate the register number using a regular expression.
4. Validate the institutional email address.
5. Validate the course code.
6. Validate the semester number.
7. Validate the mobile number.
8. Display validation messages for each field.
9. If all fields are valid, display **Registration Successful**; otherwise, display **Registration Failed**.

Python Program

```
import re

# Student Details
register_no =
"24AIDS012"
email =
"24aids012@sse.saveetha.edu.in"
course_code = "AIDS301"
semester = "5"
mobile =

"8765432109" status

= True

# Register Number Validation
```

```
if re.fullmatch(r"\d{2}[A-Z]{4}\d{3}",
   register_no): print("Register Number : Valid")
else:
   print("Register Number :
   Invalid") status = False

# Institutional Email Validation
if re.fullmatch(r"[A-Za-z0-9._%+-]+@sse\.saveetha\.edu\.in", email):
   print("Email Address  : Valid")
else:
   print("Email Address  :
   Invalid") status = False

# Course Code Validation
if re.fullmatch(r"[A-Z]{4}\d{3}",
   course_code): print("Course Code
                              : Valid")
else:
   print("Course Code  :
   Invalid") status = False

# Semester Validation
if re.fullmatch(r"[1-8]",
   semester): print("Semester
                            : Valid")
else:
   print("Semester     :
   Invalid") status = False

# Mobile Number Validation
if re.fullmatch(r"[6-9]\d{9}", mobile):
   print("Mobile Number  : Valid")
else:
   print("Mobile Number  :
   Invalid") status = False

# Final Registration Status
print("\n========== Registration Report ==========")

if status:
   print("Registration Status :
SUCCESSFUL") else:
   print("Registration Status : FAILED")
```

Sample Output



Register Number :
Valid Email Address
: Valid Course Code
: Valid
Semester : Valid
Mobile Number :
Valid

========== Registration Report ==========
Registration Status : SUCCESSFUL

Regular Expressions Used

Field	Regular Expression	Purpose
Register Number	**\d{2}[A-Z]{4}\d{3}**	**Validates register number (e.g., 24AIDS012)**
Institutional Email	**[A-Za-z0-9._%+-]+@sse\.saveetha\.edu\.in**	**Validates institutional email address**
Course Code	**[A-Z]{4}\d{3}**	**Validates course code (e.g., AIDS301)**
Semester	**[1-8]**	**Validates semester (1–8)**
Mobile Number	**[6-9]\d{9}**	**Validates a 10-digit Indian mobile number**

Result

The Student Registration Validation System was successfully implemented using Python and Regular Expressions. The program validated the student's register number, institutional email address, course code, semester, and mobile number. It displayed appropriate validation messages for each field and generated a final registration status report indicating whether the registration was successful.